UNIVERSITÀ DEL SALENTO

DiSUS DIPARTIMENTO DI Scienze Umane e Sociali

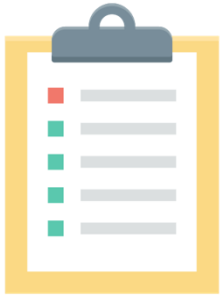
Python

Crash Course

DiSUS DIPARTIMENTO DI Scienze Umane e Sociali

Programma

- Fondamenti di Python
- Settaggio degli ambienti di sviluppo
- Virtual environment
- Object-Oriented Programming
- Analisi Dati con Pandas
- Dai Dati ai Grafici



Resources & books

On-line resources

- **Python Documentation**: https://www.python.org/doc/
- **Geek4Geek**: https://www.geeksforgeeks.org/python3-tutorial
- **DataCamp**: https://www.datacamp.com/tutorial/category/python
- **w3School**: http://www.w3schools.com/python/
- **Medium**: https://martinxpn.medium.com/100-days-of-python-9dd04d0995f1

Books

- Python Crash Course: A Hands-On, Project-Based Introduction to Programming (3rd Edition)
- Beyond the Basic Stuff with Python: Best Practices for Writing Clean Code

Python magic world

Con Python possiamo:

- analizzare dati
- automatizzare operazioni ripetitive
- costruire modelli di machine learning
- creare applicazioni web
- fare prototipi rapidamente

Perché è molto usato?

- sintassi semplice
- molto leggibile
- enorme ecosistema di librerie

Come il computer esegue il codice

- Il computer **non capisce Python** (o altri libguaggi) **direttamente**.
- Capisce solo **linguaggio macchina**.
- Serve quindi un **programma che traduce il codice**.
- Esistono due approcci principali:
  - linguaggi **compilati**
  - linguaggi **interpretati**

Linguaggi compilati

- Esempi:
  - C
  - C++
  - Rust

- Il codice deve essere **compilato** prima di essere **eseguito**.

Code file

Compiler

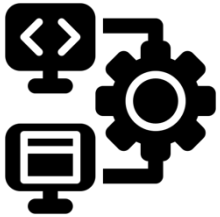
Executable file

Run

Linguaggi interpretati (Python)

- Python è un **linguaggio interpretato**.
- Il codice viene **eseguito direttamente dall'interprete**.

Code file



Interpreter

Run

Python script

- Uno **script Python** è semplicemente un file di codice
  - Estensione **.py**

```python
import numpy as np

data = [1,2,3,4]
print(np.mean(data))
```

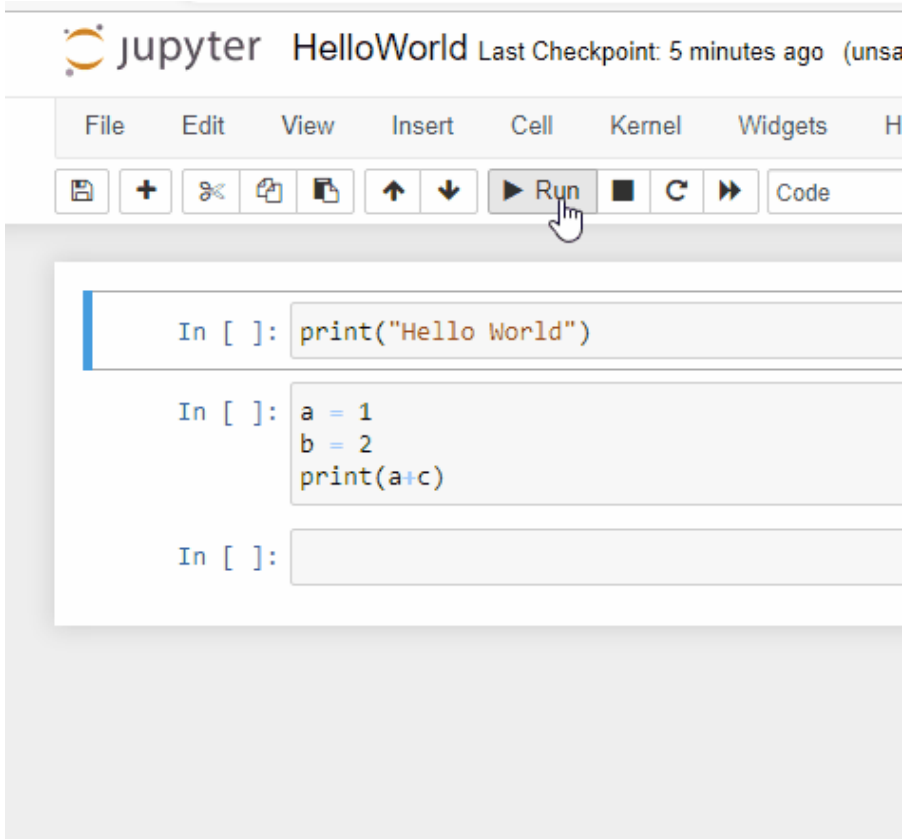
```
python analysis.py
```

Output > 2.5

- Gli script sono molto usati per:
  - automazione
  - pipeline
  - applicazioni

Cos'è un Notebook (Jupyter Notebook)

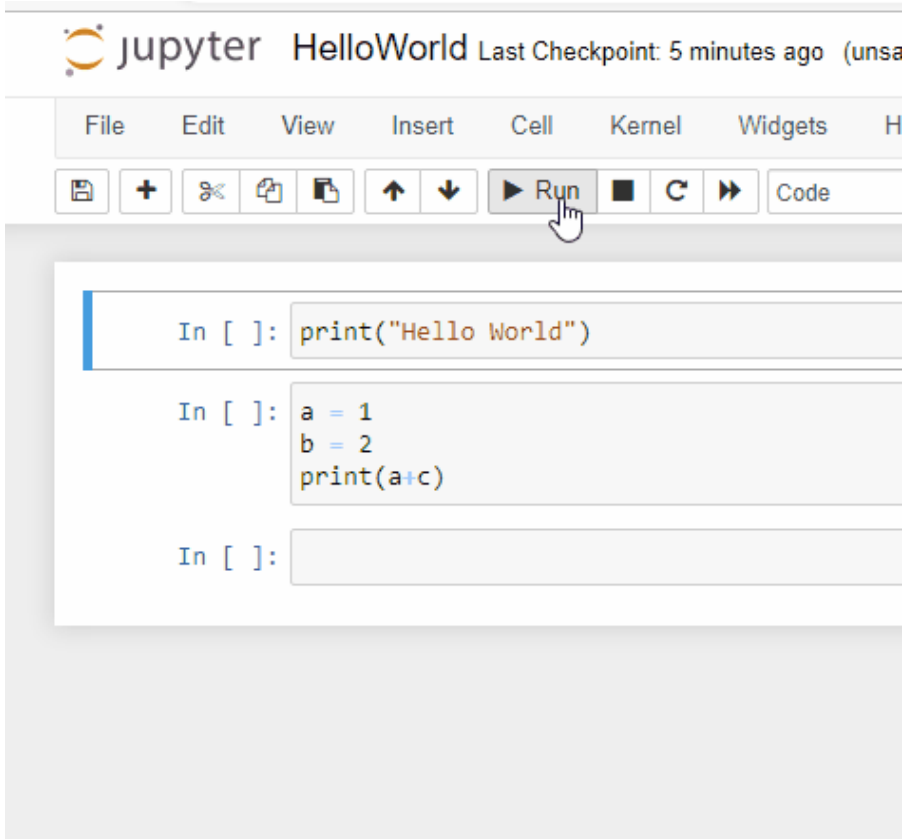
- Un notebook è un ambiente interattivo che permette di scrivere ed eseguire codice a **blocchi**, chiamati celle.
- Il notebook più usato in Python è il *Jupyter Notebook*.
- Il file ha estensione: **.ipynb**
- L'interprete lavora su ogni singola cella, come fosse un singolo file ma conserva la memoria delle celle già eseguite



Cos'è un Notebook (Jupyter Notebook)

A differenza di uno script Python (`.py`), un notebook **combina in un unico documento**:

- codice
- testo esplicativo
- grafici
- risultati delle esecuzioni
- immagini e formule



Per questo è molto usato per:

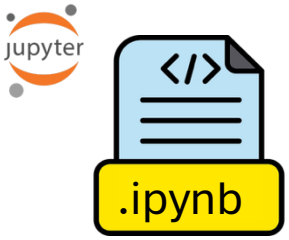
- data science
- analisi dati
- machine learning
- didattica
- report tecnici

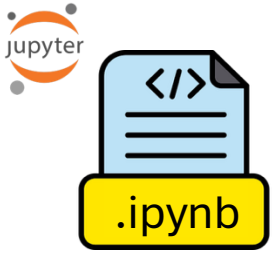
DiSUS DIPARTIMENTO DI Scienze Umane e Sociali

Strumenti

Google Colab is a hosted Jupyter Notebook service that requires no setup to use and provides free access to computing resources, including GPUs and TPUs.

Visual Studio Code is an integrated development environment (IDE) used for programming in Python.

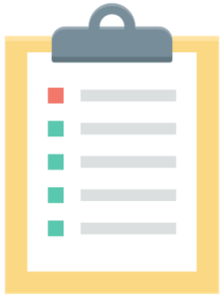




DiSUS DIPARTIMENTO DI Scienze Umane e Sociali

Day 1 – 2

- Variabili e Data Types
- Lists
- Loops
- **If** statement
- Dictionaries
- Functions



Variables

- Una **variabile** è un nome che usiamo per **memorizzare un valore**.
- In Python una variabile si crea con l'operatore **=**
- Esempio:

```
x = 10
name = "Mario"
price = 12.5
```

- Qui:
  - x contiene il valore 10
  - name contiene il testo "Mario"
  - price contiene il numero 12.5

Data types

- Ogni valore in Python ha un **tipo**.

Tipo	Descrizione	Esempio
`int`	numeri interi	`10`
`float`	numeri con decimali	`3.14`
`str`	testo	`"hello"`
`bool`	vero/falso	`True, False`

```
age = 25          # int
height = 1.75     # float
name = "Anna"     # string
is_student = True  # boolean
```

String Formatting

- La formattazione delle stringhe permette di inserire valori dentro una frase.
- È utile per creare messaggi dinamici.

- Metodo**: f-strings**
- Le **f-strings** permettono di inserire variabili direttamente nella stringa usando {}.

```
name = "Anna"
age = 25

message = f"My name is {name} and I am {age} years old"
print(message)
```

```
> My name is Anna and I am 25 years old
```

- Metodo %

Esercizi

- Ex 1
  - Traccia: Scrivi una codice che:
    - riceve nome (stringa), età (int), altezza (float in metri)
    - Crea e stampa una stringa formattata con del tipo:
    - "Mi chiamo Marco, ho 30 anni e sono alto 1.75 m"
    - Arrotonda l'altezza a 2 decimali.
  - Input: Marco, 41, 1.71345 - Output: Mi chiamo Marco, ho 41 anni e sono alto 1.71 m

- Ex 2
  - Traccia: Scrivi un codice che
    - riceve:
      - nome_file: stringa (es. "video.mp4")
      - dimensione_mb: float o int, (dimensione del file in MB)
      - velocita_mbps: float o int, (velocità di download in Mbps)
    - calcola il tempo di download in secondi usando la formula semplificata:
    - 1 byte ≈ 8 bit, quindi tempo (s) ≈ (dimensione_mb * 8) / velocita_mbps
    - restituisce una stringa formattata (puoi usare f-string o .format()) del tipo:
  - Input: "video.mp4", 100, 30
  - Output "Per scaricare video.mp4 servono circa 26.67 secondi"

Lists

- **Cos'è una lista?**
  - Una **lista** è una struttura dati che permette di **memorizzare più elementi in una singola variabile**.
  - Gli elementi sono racchiusi tra **parentesi quadre []** e separati da virgole.
- Esempio:

```
numbers = [10, 20, 30, 40]
```

Accesso alle liste

- Ogni elemento della lista ha una **posizione (indice)**.

- In Python l'indice parte da **0**

```
numbers = [10, 20, 30, 40]

print(numbers[0])   # 10
print(numbers[2])   # 30
```

Ciclo for in Python

- Un **ciclo for** serve per **ripetere un'operazione su una sequenza di elementi (**liste, stringhe, range di numeri
- Permette di **automatizzare operazioni ripetitive** senza scrivere lo stesso codice più volte.

- Esempio **liste**

```
numbers = [10, 20, 30]

for n in numbers:
    print(n)
```

```
10
20
30
```

- Esempio **range()**

```
for i in range(5):
    print(i)
```

```
0
1
2
3
4
```

Condizioni in Python: if, elif, else

- Le **istruzioni condizionali** permettono al programma di **eseguire codice diverso a seconda di una condizione**.

```
if condizione:
    istruzioni
elif altra_condizione:
    istruzioni
else:
    istruzioni
```

```
temperature = 18

if temperature > 25:
    print("Hot")
elif temperature > 15:
    print("Mild")
else:
    print("Cold")
```

- if → prima condizione da controllare
- elif → altre condizioni possibili
- else → eseguito se nessuna condizione è vera

```
> Mild
```

Operatori di confronto

Operatore	Significato
>	maggiore
<	minore
>=	maggiore o uguale
<=	minore o uguale
==	uguale
!=	diverso

Dizionari in Python (dict)

- Un **dizionario** è una struttura dati che memorizza informazioni sotto forma di **coppie chiave → valore**.
- Si usa quando vogliamo associare un **nome (chiave)** a un **dato (valore)**.
- I dizionari sono definiti con **parentesi graffe {}**.

```python
person = {
    "name": "Anna",
    "age": 25,
    "city": "Rome"
}
```

```
chiave → valore
name   → "Anna"
age    → 25
city   → "Rome"
```

Dizionari in Python (dict)

- **Accesso ai valori**
  - Possiamo accedere ai dati usando la **chiave**.

```
print(person["name"])
print(person["age"])
```

```
Anna
25
```

- **Aggiungere o modificare valori**

```
person["age"] = 26
person["job"] = "Engineer"
```

Function

- **Definizione**: Blocchi di codice riutilizzabili che eseguono un'attività specifica

Nome della funzione

Parametri di input

```
def nome_funzione(parametro1, parametro2):
    # Corpo della funzione
    risultato = parametro1 + parametro2
    return risultato

# Chiamata della funzione
somma = nome_funzione(5, 3)
print(somma)  # Output: 8
```

Output

No function style

```
price1 = 100
discount1 = 10
final_price1 = price1 - (price1 * discount1 / 100)
print("Product 1 final price:", final_price1)

price2 = 250
discount2 = 10
final_price2 = price2 - (price2 * discount2 / 100)
print("Product 2 final price:", final_price2)

price3 = 80
discount3 = 10
final_price3 = price3 - (price3 * discount3 / 100)
print("Product 3 final price:", final_price3)
```

Function style

```
def calculate_final_price(price, discount):
    final_price = price - (price * discount / 100)
    return final_price

print("Product 1 final price:", calculate_final_price(100, 10))
print("Product 2 final price:", calculate_final_price(250, 10))
print("Product 3 final price:", calculate_final_price(80, 10))
```

Funcions: Excersize

- **Ex1**
  - **Traccia:** Scrivi una funzione prezzo_scontato(prezzo, sconto_perc) che restituisce il prezzo finale dopo uno sconto percentuale.
  - **Input di test:** prezzo=80, sconto_perc=25 - **Output**: 60

- **Ex2**
  - **Traccia:** Scrivi conta_vocali(testo) che ritorna quante vocali ci sono (a,e,i,o,u), ignorando maiuscole/minuscole.
  - **Input di test:** testo="Ciao Marco". - **Output**: 4

- **Ex3**
  - **Traccia:** Scrivi stats(lista) che ritorna una tupla (minimo, massimo, media).
  - **Input di test:** lista=[2, 4, 4, 10] – **Output**: [2, 10, 5.0]

- **Ex4**
  - **Traccia:** Scrivi minmax_norm(lista) che restituisce una nuova lista normalizzata tra 0 e 1 usando:

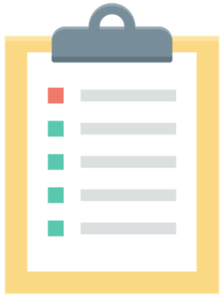
$$x' = \frac{x - min}{\min - min}$$

  - Se tutti i valori sono uguali (max==min), restituisci una lista di zeri.
  - **Input di test:** lista=[10, 20, 30] – **Output**: [0.0, 0.5, 1.0]

DiSUS DIPARTIMENTO DI Scienze Umane e Sociali

Day 3

- Object-oriented programming (OOP) Lists
- Class
- Encasulation
- Inheritance

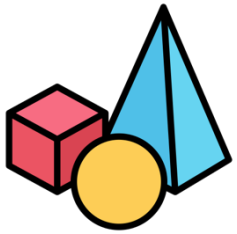


OOP: Object Oriented Programming

Object-oriented programming (OOP) o Programmazione orientate agli Oggetti é un *paradigma di programmazione* basato sul concetto di **oggetti** contenenti dati and codice.

- Dati in forma di campi (**attributes**)
- Codice in forma of procedure (**methods**).

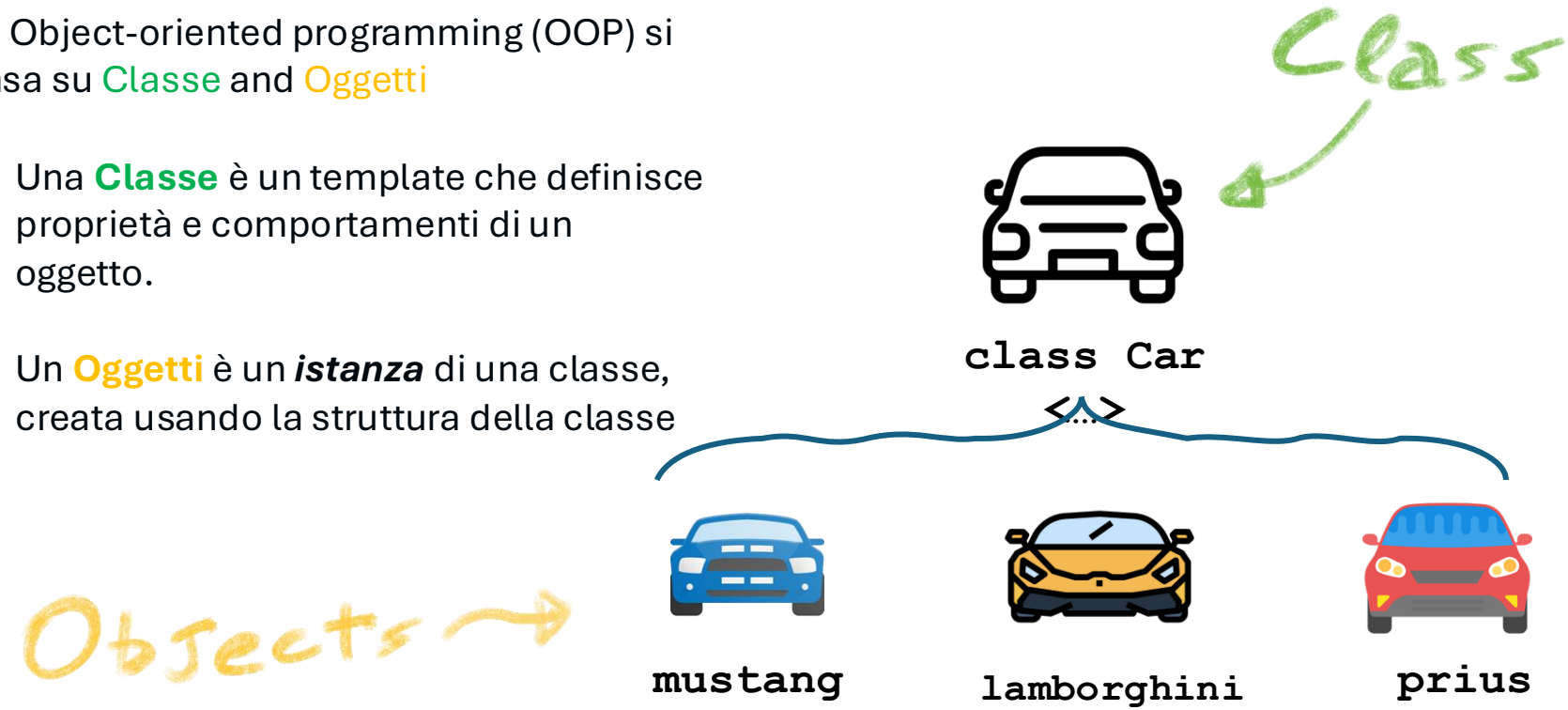
Nell OOP, I programmi sono strutturati per essere composti da oggetti che interagiscono con altri oggetti.



OOP: Classes and Objects

La Object-oriented programming (OOP) si basa su Classe and Oggetti

- Una **Classe** è un template che definisce proprietà e comportamenti di un oggetto.
- Un **Oggetti** è un ***istanza*** di una classe, creata usando la struttura della classe



OOP: Class Example

Le **Classi** sono fatte principalmente da ***Attribbuti*** e ***Metodi.***

*Per definire una classe utilizziamo la parola chiave **class** seguita dal nome della classe*

Gli **attribbuti** di una classe sono le variabili e all'interno della classe sono preceduti dal prefisso ***self.***

I **metodi** sono le funzioni che fanno parte di una classe, anche loro all'interno della classe sono preceduti dal prefisso ***self.***

Il metodo __init__() è un metodo speciale che Python esegue automaticamente ogni volta che si crea una nuova istanza basata sulla classe Dog.

__init__() non ha mai un **return**

```
class Dog():
    """A simple attempt to model a dog."""
    def __init__(self, name, age):
        """Initialize name and age attributes."""
        self.name = name
        self.age = age


    def sit(self):
        """Simulate a dog sitting in response to a command."""
        print(self.name.title() + " is now sitting.")

    def roll_over(self):
        """Simulate rolling over in response to a command."""
        print(self.name.title() + " rolled over!")
```

OOP: Attributi

Gli **attributi** sono variabili associate a un oggetto

La documentazione Python descrive gli attributi come "qualsiasi nome seguito da un punto"

Ogni oggetto ha il proprio insieme di attributi.

my_dog.name → Carlie
your_dog.name → Freddy

È possibile accedere e impostare questi attributi come qualsiasi altra variabile.

1 my_dog.name → Carlie
2 my_dog.name = Bob
3 my_dog.name → Bob

OOP: Class Example

Una volta definite una classe è possibile crearne una o più istanze

Una volta creata un'istanza è possibile utilizzare il . per

- utilizzarne i metodi
- accedere agli attributi

```
my_dog = Dog("Charlie", 7)
your_dog = Dog("Freddy", 3)

my_dog.sit()
your_dog.sit()

print(my_dog.name)
```

```
Charlie is now sitting.
Freddy is now sitting.
Charlie
```

OOP: Metodi, *__init__()*, and *self*

I **metodi** sono funzioni associate agli oggetti di una particolare classe.

Gli **oggetti** (istanze di una classe) sono creati chiamando il nome della classe come fosse una funzione. Questa funzione viene chiamata costruttore.

La chiamata del costruttore fa sì che Python crei il nuovo oggetto e poi il metodo *__init__()* con cui si *impostano i valori iniziali degli attributi*.

```
class Dog():
    def __init__(self, name)
        self.name=name

dog = Dog("Billy")
```

Tutti i metodi hanno un primo parametro chiamato **self**. <u>Quando un metodo viene richiamato su un oggetto, l'oggetto viene automaticamente passato al parametro self.</u>

Exercise ----------

Creiamo una classe Correntista

La classe dovrà contenere:

Attributi:

- Nome
- Cognome
- Saldo_disponibile
- Saldo_investito

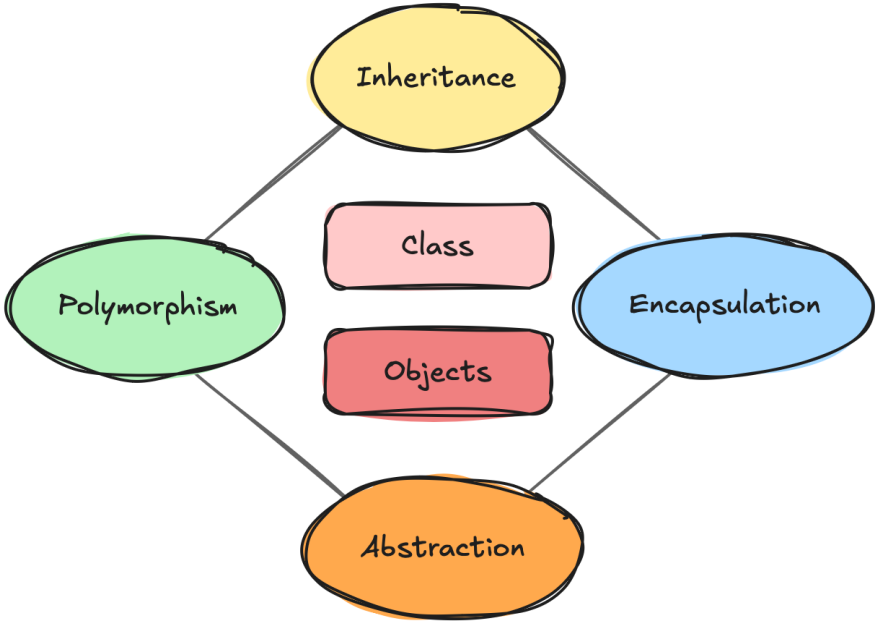
Metodi:

- preleva(somma_prelevata)
- versa(somma_versata)
- investi(somma_da_investire)

The OOP pillars: *overview*

La *Object-oriented programming* si basa su 4 pilastri fondamentali

- Encapsulation: si riferisce all'idea di nascondere i dettagli implementativi di un oggetto
- Inheritance: è un meccanismo che consente a una classe di ereditare proprietà e metodi da un'altra classe, detta superclasse o classe madre.
- Polymorphism è la capacità di un oggetto di assumere più forme.
- Abstraction: è il concetto di mostrare all'esterno solo le informazioni necessarie, nascondendo i dettagli non necessari.



The OOP pillars: *Encapsulation*

Encapsulation consists of **hiding** the implementation details of an object from the outside world and only exposing the necessary.

By using **private** and **protected** fields, developers can control the visibility and accessibility of class ***attributes*** and ***methods***, preventing accidental or intentional misuse.

Python supports encapsulation through **conventions** and **programming practices** rather than enforced access modifiers (like C, C++).

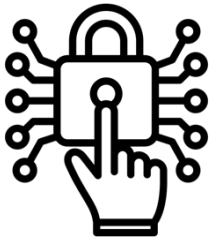
Python relies on the “We are all adults here” idea, so you can only implement encapsulation as a mere convention.

*Someone say **Protected** and **Private** ----------*

- **Public Access Modifier**: i metodi e i campi pubblici sono accessibili direttamente da qualsiasi classe.

- **Protected Access Modifier** (_): i metodi e i campi protetti **dovrebbero** essere accessibili all'interno della stessa classe dichiarata e della sua sottoclasse.

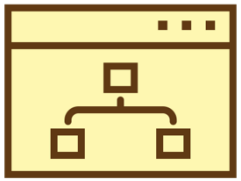
- **Private Access Modifier** (__): i metodi e i campi privati **dovrebbero** essere accessibili solo all'interno della stessa classe dichiarata. → name mangling



The OOP pillars: *Inheritance* ----------

Inheritance è una tecnica di riutilizzo del codice che si può applicare alle classi.

È l'atto di mettere le classi in una relazione genitore-figlio in cui la classe figlia eredita una copia dei metodi della classe genitrice, evitando di duplicare un metodo in più classi.



L'ereditarietà è a volte sopravvalutata o addirittura pericolosa, a causa della complessità aggiuntiva che grandi reti di classi ereditate aggiungono a un programma. -→ ***Inheritance is evil***

Tuttavia, un *utilizzo consapevole* può essere fondamentale nell'ottenere un codice:

- Ordinato
- Manutenibile
- Riusabile

Utilizzo consapevole → ***Esperienza*** → *Sbaglia, Impreca, Correggi*

DiSUS DIPARTIMENTO DI Scienze Umane e Sociali

The OOP pillars: *Inheritance* ----------

Proviamo a scrivere una scrivere una classe padre e delle classi figlie

```
class ParentClass:
  def printHello(self):
    print('Hello, world!')

class ChildClass(ParentClass):
  def someNewMethod(self):
    print('ParentClass objects don't have this method.')

class GrandchildClass(ChildClass):
  def anotherNewMethod(self):
    print('Only GrandchildClass objects have this method.')
```

The *super()* function ----------

Il metodo sovrascritto (**overridden method**) di una classe figlio è spesso simile al metodo della classe genitore.

Anche se l'ereditarietà è una tecnica di riutilizzo del codice, la sovrascrittura di un metodo potrebbe causare la riscrittura dello stesso codice del metodo della classe genitore come parte del metodo della classe figlio.

Per evitare questa duplicazione di codice, la funzione built-in **super()** consente a un metodo sovrascritto di richiamare il metodo originale della classe genitore.

super().__init__() si assicura che il costruttore della classe genitore sia chiamato, inizializzando tutti gli attributi definiti nella classe genitore. È buona norma chiamare il costruttore della classe genitore, se si tratta di un'inizializzazione importante.

Static Methods

Gli **Static Method** sono metodi di una classe che:

- Sono identificati con il *decoratore* **@staticmethod**
- Non hanno un parametro **self**
- Non hanno un parametro **cls**
- Non possono accedere ad attributi e metodi della classe

```
class ExampleClassWithStaticMethod:

  @staticmethod
  def sayHello():
    print('Hello!')

# Note that no object is created, the class name precedes sayHello():
ExampleClassWithStaticMethod.sayHello()
```

<u>Sono una tipologia di metodo che è preferibile evitare.</u>

GIT

- Git è uno strumento che permette di salvare **la storia di un progetto**, tornare a versioni precedenti e coordinare il lavoro di più sviluppatori senza perdere le modifiche

- Git serve per:
  - tracciare ogni modifica fatta al codice o ad altri file testuali;
  - confrontare versioni diverse dello stesso progetto;
  - lavorare in team senza sovrascrivere facilmente il lavoro degli altri;
  - recuperare versioni precedenti in caso di errore.

GIT vs GitHUB

- Git e GitHub non sono la stessa cosa: **Git è lo strumento di versionamento**, mentre **GitHub è una piattaforma online che ospita repository Git e aggiunge funzioni collaborative**.